PHP: Desenvolvimento Dinâmico com Estruturas de Repetição (Loops) e Coleções (Arrays)

1. O Desafio da Escalabilidade no Desenvolvimento de Software

No ensino da Engenharia de Software, observamos que a maturidade de um sistema é medida pela sua capacidade de lidar com o crescimento sem a necessidade de reescrita estrutural. O "Código Legado" inicial de um projeto muitas vezes apresenta uma rigidez perigosa: dados fixos em variáveis isoladas como $nota1, $nota2 e $nota3.Essa abordagem representa uma limitação insustentável. Imagine um cenário de "Escala Caótica" onde um filme no sistema *Screen Match* receba centenas de avaliações. Criar manualmente centenas de variáveis é um erro técnico que impede a automação e degrada a manutenção. O objetivo desta unidade é realizar a transição do código estático para o dinâmico, permitindo que o software se adapte ao volume de dados de entrada de forma resiliente. Para isso, abandonaremos as constantes no código e passaremos a utilizar a interface de linha de comando para alimentar nosso sistema.

2. Interface de Linha de Comando e Gestão de Argumentos

O terminal (CLI - *Command Line Interface*) é uma ferramenta essencial para o desenvolvedor, servindo como uma interface de entrada de dados rápida e eficiente. No PHP, a interação com o ambiente externo ocorre através de variáveis superglobais que gerenciam argumentos. É fundamental distinguir dois conceitos: o **$argc** (*Argument Count*), que é um contador do tipo inteiro, e o ****$ **argv** (*Argument Vector*), que é o container (array) contendo as strings passadas. O PHP sempre contabiliza o nome do arquivo executado como o primeiro argumento (Índice 0).

Quadro Explicativo: Anatomia dos Argumentos: **php screen-match.php 10 9 8**

Índice	Conteúdo	Papel Lógico
0	`screen-match.php`	Identificador do Script (Ignorado no processamento de dados)
1	`10`	Primeiro dado de entrada (Argumento 1)
2	`9`	Segundo dado de entrada (Argumento 2)
3	`8`	Terceiro dado de entrada (Argumento 3)
Total ($argc)	**4**	Contador total de elementos na execução

Com a compreensão do volume de dados via $argc, o próximo passo lógico é implementar estruturas que percorram esses argumentos de forma cíclica.

3. Estruturas de Controle: A Lógica da Repetição

Estruturas de repetição são essenciais para a economia de código e precisão algorítmica. Elas evitam a redundância e garantem que a mesma lógica seja aplicada a ‘n’ elementos de forma consistente.

- **Estrutura for :** É a repetição contada, ideal quando conhecemos o limite do processamento. Sua assinatura exige Inicialização, Condição e Incremento. No processamento de argumentos, iniciamos o

contador em 1 ($i = 1) propositalmente para saltar o nome do arquivo. A condição $i < $argc funciona como um **boundary check** (verificação de limite); como os índices são baseados em zero, o contador total sempre será uma unidade maior que o último índice de dado.

- **Estrutura while :** Baseia-se em uma condição booleana. O bloco é executado enquanto a expressão for verdadeira. É a escolha técnica superior para cenários de incerteza, como a leitura de fluxos de dados ou arquivos onde o fim não é conhecido previamente.

Comparativo Técnico: For vs. While

Estrutura,Funcionamento Lógico,Caso de Uso Ideal
For,Iteração com controle de estado e limites definidos.,Percorrer coleções com tamanho conhecido (ex: argumentos CLI).
While,Execução condicionada à validade de uma expressão.,Processamento de fluxos contínuos ou leitura de arquivos (EOF).
Dominada a repetição, precisamos de contêineres que organizem esses dados processados: as coleções.

4. Coleções de Dados: Arrays Indexados e Associativos

A evolução do armazenamento de software exige a transição de variáveis dispersas para **Arrays** . No PHP moderno, utilizamos a sintaxe curta de colchetes para criar esses contêineres de dados.Os arrays podem ser **Indexados** (numéricos) ou **Associativos** . Nestes últimos, substituímos índices numéricos por chaves textuais, conferindo significado semântico aos dados. Veja o exemplo de modelagem de um filme:

```
$filme = [
    'nome' => 'Top Gun: Maverick',
    'ano' => 2022,
    'nota' => 8.8
];
```

Para depurar essas estruturas, o desenvolvedor utiliza o var_dump(), o "Raio-X" das variáveis. Ele revela informações críticas que o código em execução normalmente oculta, como o **tamanho (size)** do array e o **comprimento (length)** das strings. Ao inspecionar o array acima, o var_dump indicaria array(3) e, para o nome, string(17) "Top Gun: Maverick".

5. Manipulação Dinâmica: Cast e Append

Um desafio comum em Engenharia de Software é a disparidade de tipos de dados. Entradas via terminal chegam ao PHP invariavelmente como strings. Para realizar cálculos, é imperativo aplicar o **Type Casting** (conversão de tipo).Ao alimentar um array dinamicamente, utilizamos a técnica de **Append** . O PHP gerencia o próximo índice disponível automaticamente, o que simplifica o código e evita erros de sobreposição.

```
// Conversão (Cast) e Inserção automática (Append)
$notas[] = (float) $argv[$i];
```

Este processo garante a sanitização mínima necessária, transformando a entrada textual em um número decimal (float) apto para operações aritméticas.

6. A Elegância da Iteração com foreach e Funções Nativas

O PHP oferece abstrações de alto nível que aumentam a legibilidade e reduzem a carga cognitiva do desenvolvedor. A estrutura foreach elimina a necessidade de gerenciar contadores manuais e condições de parada, focando puramente no dado.Além disso, a Biblioteca Padrão do PHP (SPL) fornece funções que encapsulam lógicas complexas. O uso de funções nativas como array_sum() é um divisor de águas entre o código júnior (manual e prolixo) e o código sênior (eficiente e performático).

Evolução da Legibilidade: Soma de Coleções

Antes (Lógica Manual),Depois (Função Nativa)
foreach ($notas as $nota) {,$soma = array_sum($notas);
$soma += $nota;,
},

7. Arquitetura do Fluxo: Screen Match v2

O projeto evolui para uma arquitetura de fluxo contínuo, onde o sistema é capaz de processar milhares de notas com a mesma base de código. O fluxo do **Screen Match v2** organiza-se em quatro pilares:

1. **Input CLI:** Captura de argumentos brutos através do terminal.
2. **Processamento:** Uso de loops (for) para percorrer o $argv, aplicando **Cast** para tipagem correta.
3. **Cálculo:** O Array atua como um **Array Storage** (buffer temporário) que é então processado pela função array_sum().
4. **Output:** Entrega da informação final (Média) ao usuário.

8. Perspectivas Futuras: Introdução à Orientação a Objetos

Embora tenhamos dominado a lógica estruturada e as coleções, o próximo nível de maturidade envolve modelar elementos do mundo real como entidades independentes. Entramos no paradigma da **Programação Orientada a Objetos (POO)** .Nesta nova etapa, estruturaremos o código através de:

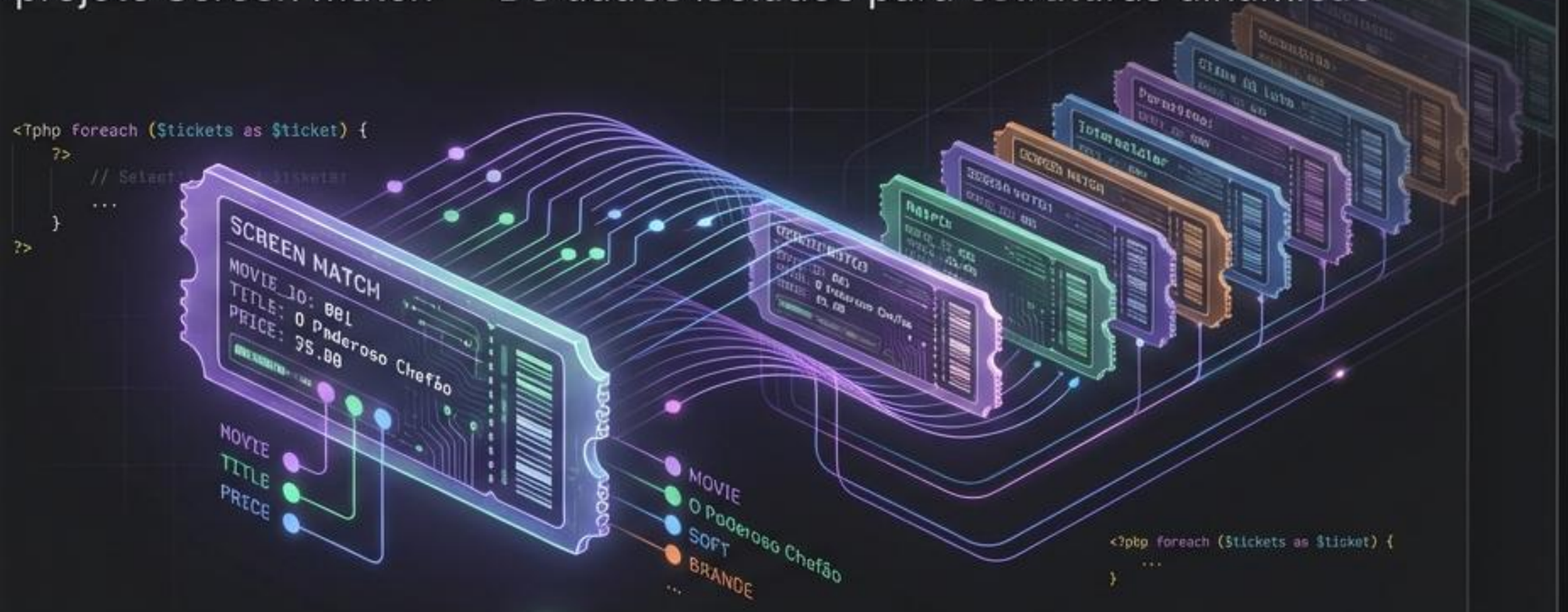
- **Classes:** O projeto ou molde técnico.
- **Objetos:** As instâncias vivas na memória baseadas no molde.
- **Atributos:** Dados e características (ex: o título do filme).
- **Métodos:** Comportamentos e ações (ex: o cálculo da média do filme).

9. Glossário Técnico

- **$argc:** (*Argument Count*) Variável inteira reservada que armazena o número total de argumentos passados via CLI, incluindo o nome do arquivo.
- **$argv:** (*Argument Vector*) Array de strings contendo todos os argumentos fornecidos na linha de comando. Por padrão, todos os dados de entrada são tratados como strings até que um cast seja realizado.
- **Array Associativo:** Coleção onde os valores são mapeados para chaves nomeadas (strings), permitindo uma organização semântica dos dados.
- **Cast (Tipagem):** Conversão explícita de um valor de um tipo para outro (ex: (float) para converter string em decimal). Necessário para garantir a integridade em operações aritméticas.
- **foreach:** Estrutura de iteração de alto nível especializada em percorrer coleções sem a necessidade de índices manuais.
- **Iteração:** O processo de repetição de um bloco de instruções dentro de um ciclo algorítmico.
- **var_dump:** Função de diagnóstico fundamental para inspeção de variáveis; exibe o tipo de dado, o valor e metadados como tamanho de coleções e comprimento de strings.

PHP: Dominando Loops e Coleções

Evoluindo o projeto Screen Match — De dados isolados para estruturas dinâmicas



Curso de Desenvolvimento de Sistemas - 3º Ano

NotebookLM

O Problema da Escala (Contexto Semana 01)

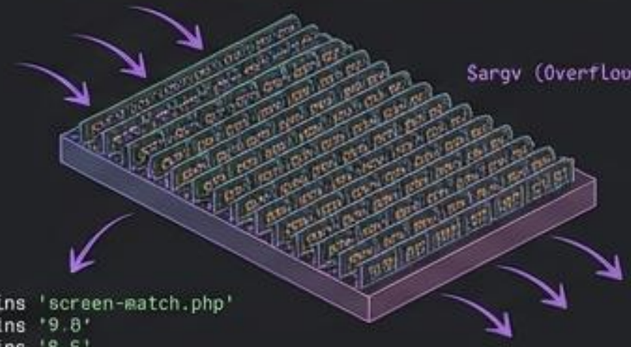
Código Legado (Fixo)

```php
<?php

    $nota1 = 9.0;
    $nota2 = 8.5;
    $nota3 = 7.0;
;
```

Terminal (Escala Caótica)

```
>_ php screen-match.php 9.0 8.5 7.0 6.5 9.2 8.8 7.9 9.5 8.1 7.3 9.9 8.4 7.1 6.8
9.7 8.6  8.6 7.8 9.5 8.0 7.5 9.1 8.7 7.2 9.8 8.3 7.4 9.6 8.2 7.7 9.4 8.9 7.6
10.0 6.7 9.0 8.5 7.0 6.5 9.2 8.8 7.9 9.5  8.1 7.3 9.9 8.4 7.1 6.8 9.7 8.6...
```




```
Index [0] contains 'screen-match.php'
Index [1] contains '9.0'
Index [2] contains '8.5'
Index [3] contains '8.0'
Index [4] contains '8.3'
Index [5] contains '9.2'
```

O código atual é rígido. Se o filme receber 100 avaliações, precisaremos criar 100 variáveis? **O objetivo de hoje é tornar o código dinâmico.**

NotebookLM

Conceito - argc

Definindo o Limite da Repetição

```
php screen-match.php 10 9 8
```

Index 0 | Argumento | Argumento | Argumento

```
$argc = 4
```

Argument Count

```
var_dump($argc); // int(4)
```

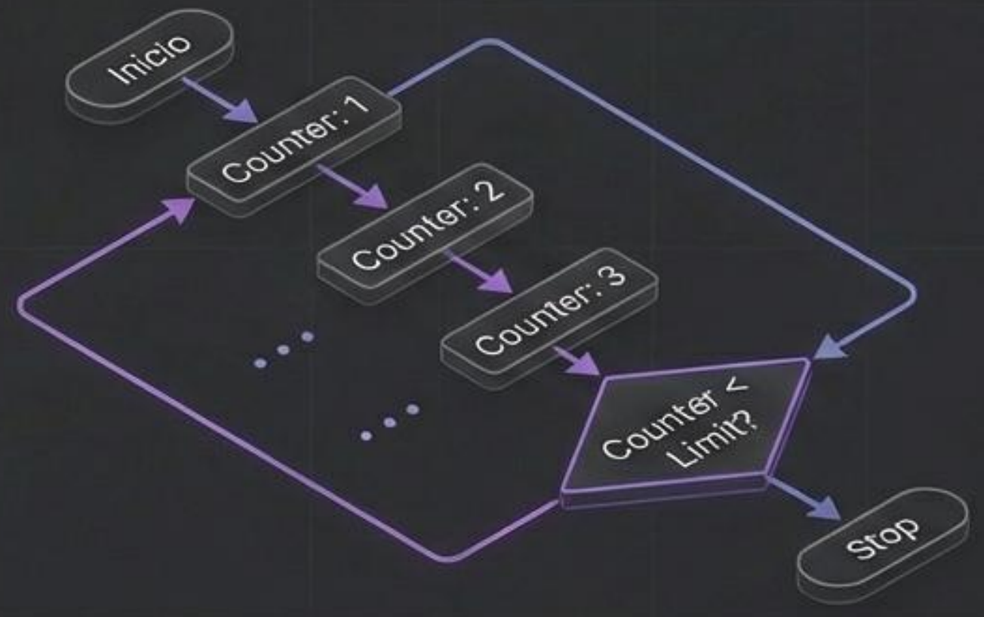
A variável **$argc** (Argument Count) armazena a quantidade total de argumentos. O nome do script conta como o primeiro item.

NotebookLM

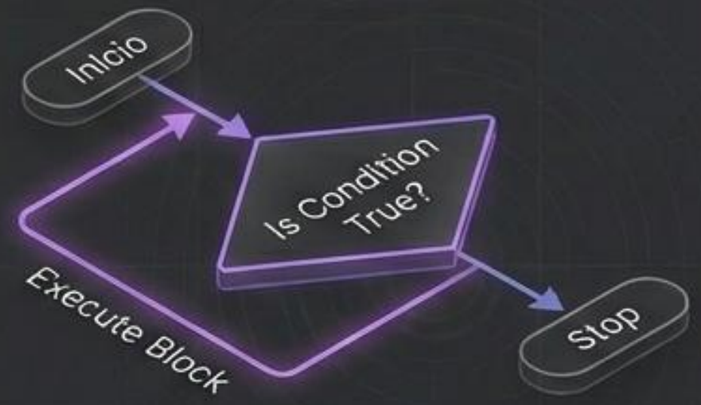
Conceito - while

Repetição Baseada em Condição

FOR



WHILE



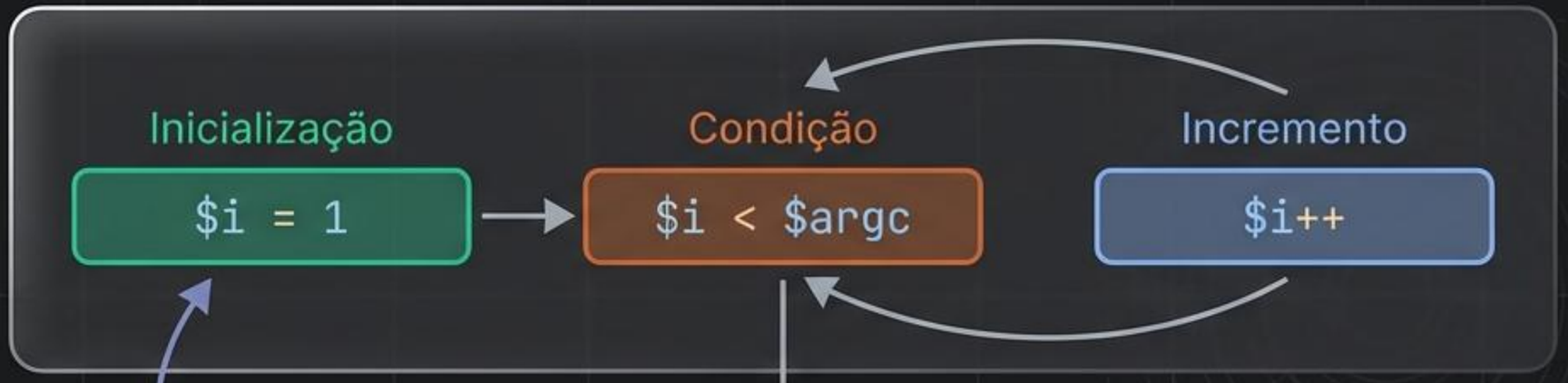

```php
<?php
while ($condicao == true) {
  // Executa o bloco enquanto verdadeiro
}
?>
```

Ideal para quando não sabemos o número exato de repetições (ex: ler um arquivo até o fim).

NotebookLM

Conceito - for

A Estrutura de Repetição Contada



Iniciamos em 1 para pular o nome do arquivo.

```php
<?php
for ($i = 1; $i < $argc; $i++) {
    // Processa a nota no índice $i
}
?>
```

NotebookLM

Conceito - Array Associativo

Dando Nome aos Índices

Keys	Values
'nome'	'Top Gun: Maverick'
'ano'	2022
'nota'	8.8

```php
$filme = [
    'nome' => 'Top Gun: Maverick',
    'ano' => 2022,
    'nota' => 8.8
];
```

Substituímos índices numéricos confusos ([0]) por chaves de texto ('nome') para dar significado aos dados.

NotebookLM

Conceito - var_dump

O Raio-X da Variável

Tipo de dado (array)

Tamanho

```
array(3) {
  ["nome"]=>
  string(17) "Top Gun: Maverick"
  ["ano"]=>
  int(2022)
  ["nota"]=>
  float(8.8)
}
```

Índice

Valor e Tipo

NotebookLM

Conceito - Sintaxe de Array

O Container de Dados

```
// Ruim (Difícil de escalar)
$nota1 = 10;
$nota2 = 8;
```

```
// Bom (Moderno e Dinâmico)
$notas = [10, 8, 9, 7.5];
```

Utilizamos a sintaxe curta [ ] moderna. O índice sempre começa em 0.

NotebookLM

Conceito - Cast & Append

Sanitização e Inserção Dinâmica

```
// Conversão e Inserção automática
$notas[] = (float) $argv[$i];
```

O PHP converte o tipo (Cast) e gerencia automaticamente o próximo índice vazio do array.

NotebookLM

Conceito - foreach()

A Elegância da Iteração

Before and After

```php
for ($i=0; $i < count($notas); $i++) {
    $nota = $notas[$i];
    // Lógica complexa de índices
}
```

Simplificação →

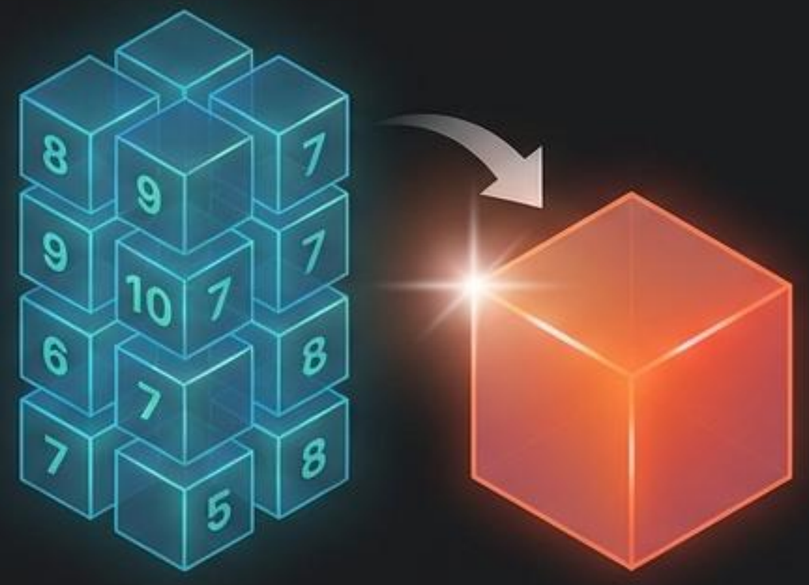
```php
foreach ($notas as $nota) {
    // Foco total no dado ($nota)
}
```

Eliminamos o gerenciamento manual de contadores.
"Para cada nota na lista de notas..."

NotebookLM

Conceito - array_sum

O Poder da Biblioteca Padrão



Array de Números

Soma Total

De/Para

```
// De:
foreach ($notas as $nota) {
    $soma += $nota;
}
```

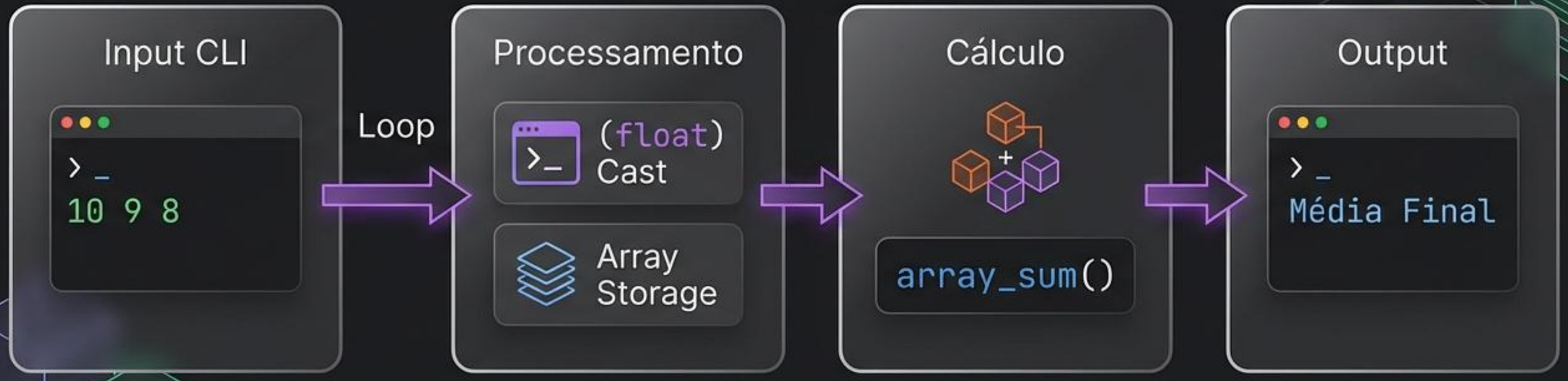
↓

```
// Para:
$soma = array_sum($notas);
```

Conhecer as funções nativas separa o código júnior do código sênior.
Menos linhas, mais legibilidade.

NotebookLM

O Fluxo Completo: Screen Match v2

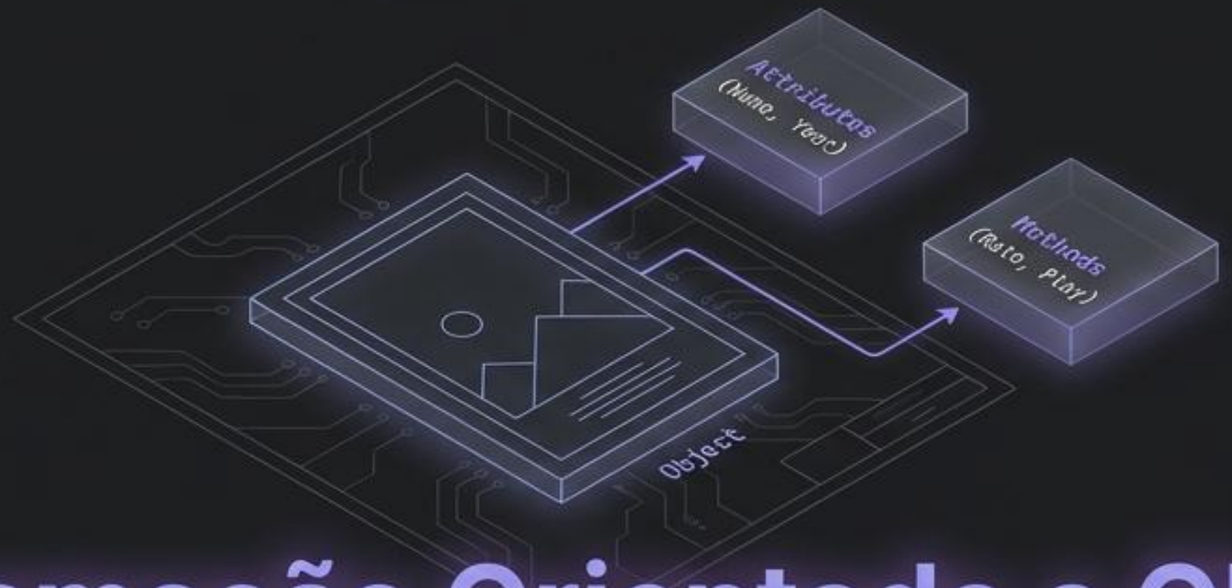


Saímos de variáveis soltas para uma estrutura capaz de processar milhares de notas com o mesmo código.

NotebookLM

Próximo Nível: Paradigmas

Modelando o Mundo Real



Programação Orientada a Objetos

- Classes
- Objetos
- Atributos e Métodos

Dominamos a lógica estruturada. Agora vamos estruturar as entidades.

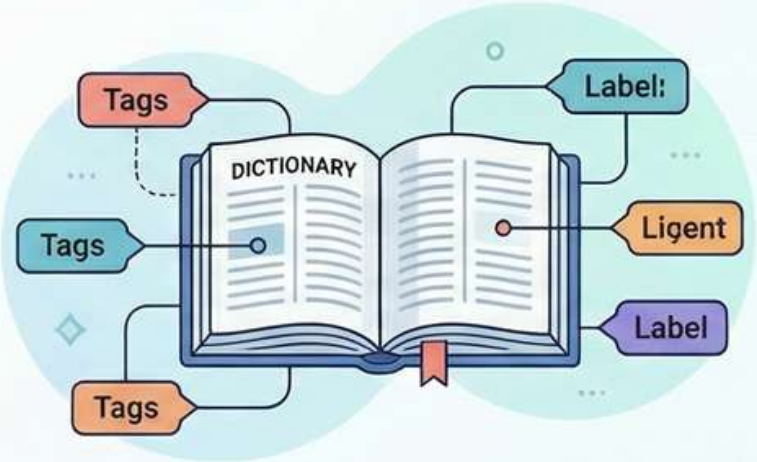
NotebookLM

PHP: Dominando Arrays Associativos

Conceito e Estrutura

O que é um Array Associativo?

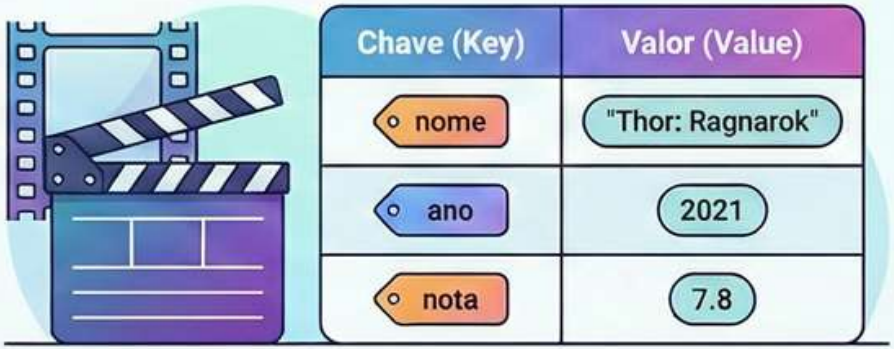
É uma coleção de dados onde cada valor está vinculado a uma chave específica (string).



Legibilidade e Organização

Substitui índices numéricos confusos (0, 1, 2) por nomes claros que descrevem o dado.

Mapeamento de Dados (Exemplo de Filme)



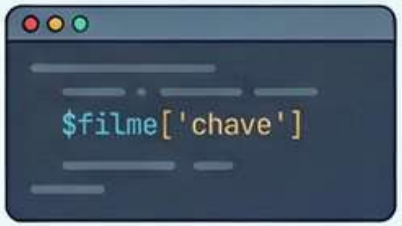
Chave (Key)	Valor (Value)
nome	"Thor: Ragnarok"
ano	2021
nota	7.8

Sintaxe e Uso Prático

Operador de Associação (=>)

Utiliza-se a "seta" para apontar uma chave para seu respectivo valor dentro dos colchetes.

Como Acessar a Informação



Basta informar o nome da chave entre colchetes, utilizando aspas simples ou duplas.

NotebookLM

PHP: Dominando a Manipulação de Arrays

Preparação e Inserção de Dados

Conversão de Tipo (Casting)

Use **(float)** para converter strings numéricas em números decimais processáveis.

Inserção Dinâmica com $array[]

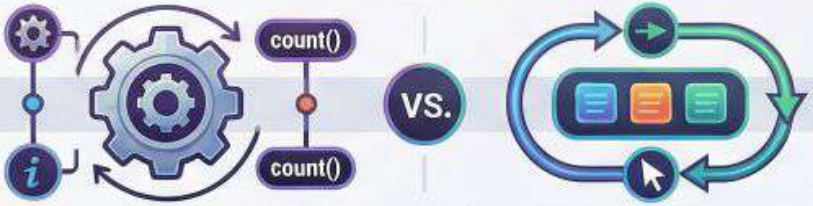
Adicione itens ao final de uma lista sem precisar gerenciar índices manualmente.

Função count()

Determine dinamicamente o tamanho total de um array para evitar "números mágicos" no código.

Iteração e Soma Eficiente

For vs. Foreach

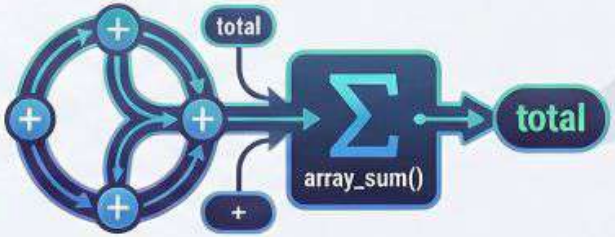


For Loop (Alta Complexidade)
Evitar para somas simples

Foreach Loop (Média Complexidade)
Bom para operações personalizadas

Simplificação com array_sum()

Substitua loops de soma manuais por esta função nativa que totaliza o array instantaneamente.



Comparação de Legibilidade e Complexidade

Loop For	Alta Complexidade (Exige contador e count)	Evitar para somas simples
Loop Foreach	Média Complexidade (Mais legível)	Bom para operações personalizadas
array_sum()	Mínima Complexidade (Função Nativo)	Ideal para totalização de valores

NotebookLM

PHP no Terminal: Dominando Argumentos e o Loop `for`

Capturando Dados via CLI ($argv e $argc)

$argv: O Array de Valores

Armazena todos as parâmetros passados no terminal, começando pelo nome do arquivo no indice zero.

$argc:

$argc: O Contador de Argumentos

Contém o número total de itens presentes no $argv, incluindo o noma de script executado.

Cálculo da Quantidade Real

4 – 1

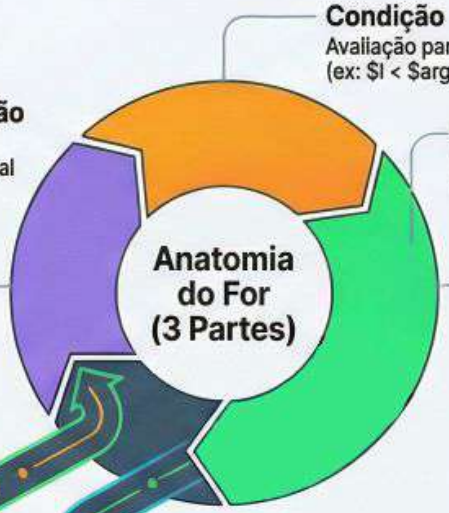
$argc (script.php)

3

(Dados do Usuário)

Subtraia 1 de $argc para obter apenas a quantidade de dados informados pelo usuário.

Processamento com o Loop `for`



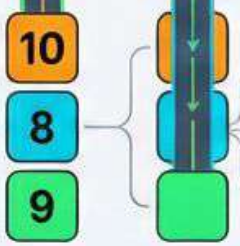
Condição
Avaliação para continuar (ex: $I < $argc)

Inicialização
Definição do contador inicial (ex: $I = 1)

Incremento
Evolução do contador (ex: $I++)

++

Operador de Incremento (++)
Atalho sintático para adicionar 1 ao valor atual da variável contadora a cada iteração.



Soma Dinâmica de Notas

Loop 1: Soma = 0 + 10

Loop 2: Soma = 10 + 8

Loop 3: Soma = 18 + 9

27

O loop percorre os indices do $argv para somar valores de forma automática e eficiente.

NotebookLM

PHP: Dominando Loops Condicionais (While & Do-While)

Diferente do loop **for**, que é baseado em um número fixo de repetições, as estruturas **while** e **do-while** focam na intenção e na condição. Elas são ideais para processar dados de tamanho desconhecido, como a leitura de arquivos ou entradas de usuários.

O Loop While (Enquanto)

Verificação Antes da Ação

O bloco de código só é executado se a condição inicial for verdadeira.

Ideal para Dados Dinâmicos

Perfeito para ler arquivos linha a linha sem saber o total de registros.

Sintaxe: while (condição) { ... }

```
while ($condicao) {
  // Código a executar;
  $incremento;
}
```

Requer atenção ao incremento manual da variável de controle para evitar loops infinitos.

O Loop Do-While (Faça... Enquanto)

Ação Antes da Verificação

O código é executado primeiro e a condição é validada apenas ao final.

Garantia de Execução Única

Diferente do while comum, o do-while sempre executa o bloco ao menos uma vez.

Uso Comum: Menus Interativos

Exibe o menu ao usuário antes de validar a opção de saída digitada.

Comparativo de Intenção

Estrutura	Momento da Verificação	Garantia de Execução
While	No início (pré-teste)	Pode nunca executar
Do-While	No final (pós-teste)	Executa pelo menos uma vez
For	No início (contador)	Baseado em limite fixo

NotebookLM

DOMINANDO ARRAYS NO PHP: A ESTRUTURA DE DADOS ESSENCIAL

Arrays são a estrutura de dados mais importante do PHP, servindo como listas ou mapas de informações. Este guia foca na utilização da sintaxe moderna com colchetes e na análise técnica de tipos e valores através da função **var_dump()**.

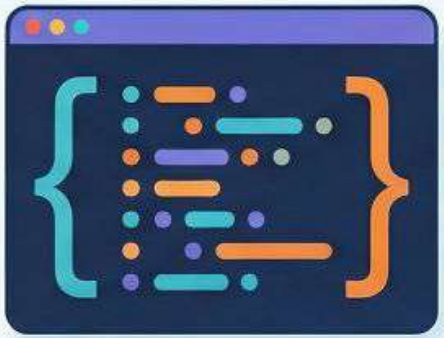
INSPEÇÃO E ESTRUTURA BASE

O Coração do PHP

É difícil encontrar código PHP real que não utilize arrays para organizar conjuntos de dados.

var_dump(): O Truque de Inspeção

Função essencial para visualizar o conteúdo, o índice e o tipo de dado de uma variável.



SINTAXE MODERNA [ ]

Embore o comando **array()** funcions, o uso de colchetes é a forma atual e preferida.

TIPOS DE ARRAYS

Arrays Indexados

```
[10, 8, 9.5]
```

Listas automáticas onde cada elemento recebe um indice numérico começando obrigatoriamente em zero.

Acesso via Colchetes

```
$variavel['chave_ou_indice']
```

Para recuperar um dado, utiliza-se a variável seguida do indice ou chave entre colchetes.

Arrays Associativos

```
['ano' => 2021]
```

Funcionam como dicionários, onde você define strings (chaves) para rotular cada valor armazenado.

COMPARATIVO DE TIPOS DE ARRAYS

	EXEMPLO:	QUANDO USAR:
Indexado	[10, 8, 9.5]	Listas simples de valores relacionados.
Associativo	['ano' => 2021]	Quando as informações precisam de nomes (rótulos).

NotebookLM